

Apuntes Semana 9

Kendell Garbanzo Calvo 2022123716

Instituto Tecnológico de Costa Rica

Inteligencia Artificial

21 de abril, 2026

I. REPASO: CLASIFICACIÓN DE MNIST

El dataset MNIST consiste en muestras de dígitos escritos a mano, donde cada imagen está representada por una matriz de píxeles. Tomando cada píxel como un *feature*, es posible construir una **regresión logística** que realice clasificación binaria de un dígito específico: el modelo aprende a reconocer o no la presencia de ese dígito en la imagen.

La regresión logística se compone de tres elementos fundamentales. Una parte **lineal**, que modela la interacción entre los features y los pesos del modelo:

$$f_{w,b}(x) = wx + b \quad (1)$$

Una parte **no lineal**, dada por la función sigmoide, que transforma la salida lineal en una probabilidad entre 0 y 1:

$$g(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

Y una **anidación** de ambas funciones, donde primero se ejecuta la transformación lineal y su resultado se pasa a la función sigmoide:

$$h(x) = g(f(x)) \quad (3)$$

Esta composición permite al modelo tomar decisiones binarias (pertenece o no a la clase) a partir de entradas de alta dimensionalidad como imágenes.

II. SISTEMA MULTINOMIAL

Un sistema multinomial extiende la clasificación binaria para manejar **múltiples clases** de forma simultánea. En lugar de responder si una entrada pertenece o no a una clase, el modelo debe determinar a cuál de N clases posibles pertenece.

II-A. Solución: One-Hot Encoding

Para representar múltiples clases se utiliza la **codificación One-Hot**: cada clase se representa como un vector binario de longitud N , donde únicamente la posición correspondiente a la clase correcta toma el valor 1 y todas las demás toman el valor 0. Esto elimina cualquier relación ordinal artificial entre las clases.

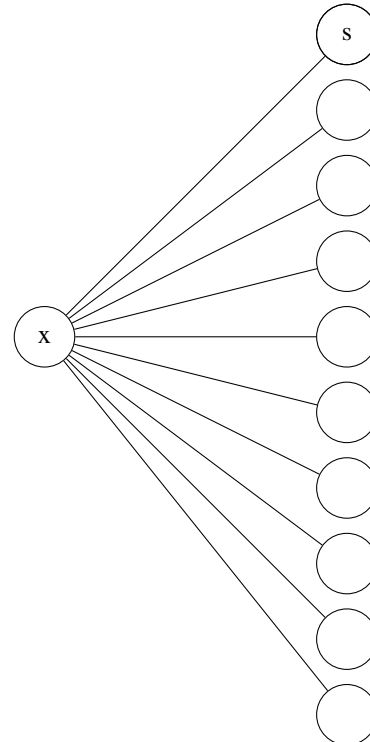
Aplicado a MNIST (dígitos del 0 al 9), la solución consiste en entrenar **10 regresiones logísticas en paralelo**, cada una especializada en reconocer un dígito específico. Durante la inferencia, toda la información de la imagen se distribuye simultáneamente a las 10 regresiones, y el resultado final es el *one-hot vector* que indica cuál regresión produjo la mayor probabilidad de activación.

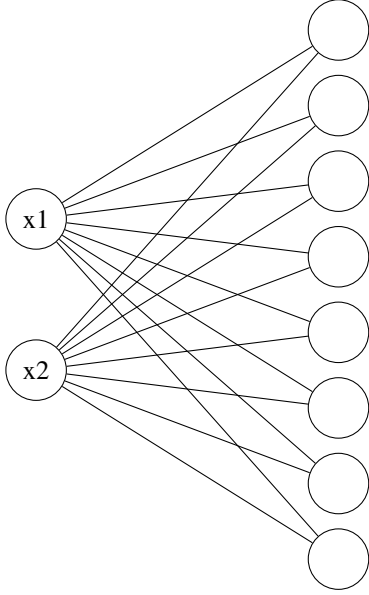
Cuadro I
CODIFICACIÓN ONE-HOT PARA DÍGITOS DEL 0 AL 9

Clase	One-Hot Codificación
0	1000000000
1	0100000000
2	0010000000
3	0001000000
4	0000100000
5	0000010000
6	0000001000
7	0000000100
8	0000000010
9	0000000001

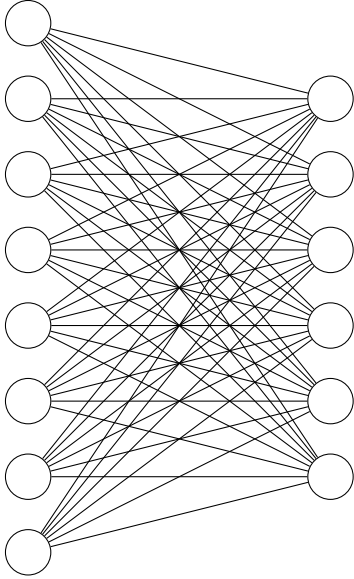
II-B. Escalamiento con la cantidad de píxeles

Con un solo píxel de entrada, cada regresión logística recibe un único valor con su peso w y su *bias* b . Al agregar un segundo píxel, cada regresión pasa a tener $2w$ y un *bias*.





Al aumentar la cantidad de píxeles el sistema escala rápidamente: con 784 píxeles la representación gráfica se vuelve imposible, pero el principio se mantiene: **toda la información de entrada se distribuye a todas las regresiones logísticas** para que cada una tome su decisión de forma independiente.



III. CONVERSIÓN DE VECTOR A MATRIZ

Calcular cada regresión logística de forma individual requiere N operaciones separadas, donde N es el número de clases. Para hacer esto eficiente se recurre al **álgebra lineal**: se reemplazan los vectores de pesos individuales por una **matriz de pesos $\mathbf{W}$** , permitiendo computar toda la capa en una sola operación matricial:

$$\mathbf{Z} = \mathbf{X} \mathbf{W} + \mathbf{b} \quad (4)$$

donde $\mathbf{X}$ es el vector de entrada, $\mathbf{W}$ contiene los pesos de todas las regresiones logísticas y $\mathbf{b}$ es el vector de *bias*, con un valor por cada regresión.

$$\mathbf{W} = \begin{matrix} & w_{0,0} & w_{0,1} & w_{0,2} & \cdots & w_{0,n-2} & w_{0,n-1} \\ w_{1,0} & w_{1,1} & w_{1,2} & \cdots & w_{1,n-2} & w_{1,n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ w_{J,0} & w_{J,1} & w_{J,2} & \cdots & w_{J,n-2} & w_{J,n-1} \end{matrix}$$

Para MNIST, con una capa de entrada de 784 píxeles y una capa de salida de 10 clases, la matriz de pesos tiene dimensiones $\mathbf{W} \in \mathbb{R}^{784 \times 10}$ (los índices comienzan en 0, por lo que se resta uno a cada dimensión). Esta representación matricial es la base del cómputo eficiente en hardware moderno como GPUs.

En el caso de MNIST, con 784 píxeles de entrada y 10 clases de salida, la matriz de pesos se representa como:

$$\mathbf{W} = \begin{matrix} & w_{0,0} & w_{0,1} & w_{0,2} & \cdots & w_{0,783} \\ w_{1,0} & w_{1,1} & w_{1,2} & \cdots & w_{1,783} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{9,0} & w_{9,1} & w_{9,2} & \cdots & w_{9,783} \end{matrix}$$

IV. CLASIFICACIÓN CON CAPAS INTERMEDIAS

Una sola capa de regresiones logísticas conectada linealmente a la entrada constituye todavía un **problema lineal**: sin importar cuántas regresiones se encadenen linealmente, el resultado sigue siendo una transformación lineal de la entrada.

La clave está en aplicar la **función sigmoide** a la salida de la capa intermedia. Al hacerlo, los valores de salida quedan en el rango $[0, 1]$ y la transformación pasa a ser **no lineal**. Esa salida no lineal es la que se pasa como entrada a la siguiente capa, que ya no opera sobre la información original sino sobre una **representación no lineal computada previamente**.

Las capas del sistema se identifican como:

- **Capa de entrada:** recibe los features originales. Su tamaño está determinado por la dimensionalidad del problema (ej. 784 para MNIST).
- **Capa intermedia:** realiza el cómputo entre la entrada y la salida. Es la responsable de introducir **no-linealidad** en el sistema, lo que permite al modelo aprender representaciones más complejas.
- **Capa de salida:** produce la decisión final, que puede ser binaria (sigmoide) o multinomial, basándose en la representación generada por la capa anterior.

V. REDES NEURONALES

Una **red neuronal** es la generalización de este sistema de capas: múltiples capas intermedias apiladas, cada una introduciendo no-linealidad adicional, permiten al modelo aprender representaciones jerárquicas de los datos.

La **no-linealidad** es el elemento fundamental que diferencia una red neuronal de una simple regresión lineal: sin ella, apilar capas no aportaría capacidad expresiva adicional. Con ella, cada capa puede aprender patrones progresivamente más abstractos.

Las características fundamentales de una red neuronal son:

- **Profundidad:** entre más capas intermedias tenga la red, más complejos son los problemas que puede resolver. La profundidad es un hiperparámetro del modelo.
- **Diversidad entre capas:** es importante que cada capa aprenda representaciones distintas para que la profundidad sea útil y no redundante.
- **Optimizabilidad:** siempre que las funciones de activación sean derivables, es posible aplicar descenso de gradiente para optimizar todos los pesos de la red de forma conjunta mediante *backpropagation*.
- **Neuronas:** cada capa está compuesta por unidades de cómputo individuales llamadas neuronas, cada una con sus propios pesos y *bias*.

EXPLICACIÓN PROYECTO I

El proyecto consiste en diseñar e implementar una solución completa de inteligencia artificial para el reconocimiento de comandos de voz, abarcando desde la selección y preprocesamiento de los datos hasta el despliegue del modelo en una aplicación móvil funcional. Se utilizará el dataset *Speech Commands Dataset v0.02* y el modelo final deberá integrarse en una aplicación Android o iOS mediante **ONNX Runtime Mobile**, realizando inferencia **localmente** en el dispositivo sin depender de servicios externos ni APIs en la nube.

Objetivo General

El objetivo es construir un sistema de clasificación de audio capaz de reconocer 10 comandos de voz específicos y desplegarlos dentro de una aplicación móvil navegable. La aplicación debe diseñarse como una experiencia de interacción completamente basada en voz, inspirada en un escenario de *Smart Home*, donde el usuario pueda desplazarse entre pantallas, controlar dispositivos simulados y confirmar acciones únicamente mediante comandos de voz. No se espera únicamente una prueba técnica del modelo, sino una experiencia funcional coherente con el caso de uso propuesto.

Comandos Autorizados

El sistema debe reconocer exactamente los siguientes 10 comandos, cada uno con un rol definido dentro de la interfaz:

Cuadro II
COMANDOS DE VOZ AUTORIZADOS Y SU CASO DE USO

Comando	Caso de uso
yes / no	Confirmación y cancelación de acciones
up / down / left / right	Navegación entre opciones de la interfaz
on / off	Control de encendido de dispositivos
stop / go	Control de inicio y detención de rutinas

Los comandos de navegación (*up*, *down*, *left*, *right*) deben utilizarse para mover el foco visual entre opciones dentro de cada pantalla. Los comandos *yes* y *no* confirman o cancelan acciones sensibles. Los comandos *on* y *off* actúan sobre el estado de dispositivos, y *go* y *stop* inician o detienen rutinas predefinidas. La coherencia entre los comandos reconocidos y su uso dentro de la aplicación es un criterio de evaluación.

Concepto de Aplicación Móvil

La aplicación debe construirse como una interfaz de navegación por voz con al menos las siguientes pantallas:

- **Dashboard:** pantalla principal con tarjetas o secciones (luces, ventilador, televisión, rutinas, configuración). El usuario navega entre tarjetas con *up/down/left/right* y abre una sección con *yes*.
- **Control de dispositivo:** permite encender o apagar un dispositivo con *on* y *off*. El comando *yes* confirma acciones destacadas y *no* regresa al dashboard.
- **Rutinas:** permite iniciar o detener rutinas predefinidas (estudio, noche, bienvenida) con *go* y *stop*. La navegación entre rutinas se realiza con *up/down*.
- **Confirmación:** pantalla que aparece ante acciones sensibles. El usuario debe decir explícitamente *yes* para ejecutar la acción o *no* para cancelar y regresar.

Dataset y Preprocesamiento

Se utilizará el *Speech Commands Dataset v0.02*, un conjunto de datos público que contiene grabaciones de corta duración etiquetadas por categoría sonora. Dado que las redes neuronales convolucionales (CNN) reciben imágenes como entrada, los audios deben transformarse en representaciones visuales antes del entrenamiento. Concretamente, cada grabación de audio debe convertirse en un **espectrograma**, que es una representación bidimensional de la señal donde el eje horizontal representa el tiempo, el eje vertical la frecuencia y la intensidad del color la amplitud de cada componente frecuencial.

Se deben preparar **dos conjuntos de datos** claramente diferenciados:

- **Conjunto crudo:** los audios se convierten directamente a espectrogramas sin ninguna modificación adicional. Este conjunto representa la línea base del experimento.
- **Conjunto aumentado:** idéntico al anterior en estructura, pero con la aplicación de técnicas de *data augmentation* inspiradas en audio. Se recomienda el uso de **SpecAugment**, que aplica enmascaramiento aleatorio de bandas de frecuencia y segmentos de tiempo directamente sobre el espectrograma. El uso de estas técnicas debe estar debidamente justificado con literatura en el informe.

Cada uno de los dos conjuntos debe dividirse en **tres particiones** independientes: entrenamiento, validación y testing. La partición de validación se utiliza durante el entrenamiento para monitorear el desempeño del modelo y detectar overfitting de forma temprana. La partición de testing se reserva exclusivamente para la evaluación final del modelo seleccionado y no debe usarse en ningún momento durante el entrenamiento ni la selección de hiperparámetros.

Diseño de Arquitectura

Se deben construir manualmente **dos modelos** con PyTorch, utilizando únicamente `torch.nn` para la definición de capas, sin librerías de alto nivel que abstraigan la arquitectura. Ambos modelos deben incluir: definición completa de la arquitectura

con su respectivo **diagrama**, función de pérdida, optimizador, rutinas de entrenamiento, validación y test, control de aleatoriedad para reproducibilidad y registro de métricas.

Modelo A — LeNet-5 clásico: debe implementarse una variante de la arquitectura clásica LeNet-5, adaptada a la clasificación de audio a partir de representaciones espectrales. Se permite modificar el tamaño de entrada, el número de canales y los hiperparámetros de las capas existentes. Cualquier otra modificación a la arquitectura original debe consultarse previamente con el profesor y justificarse en el informe. LeNet-5 es una red convolucional clásica compuesta por capas convolucionales seguidas de capas de *pooling* y capas totalmente conectadas al final, lo que la convierte en un punto de partida simple pero efectivo para tareas de clasificación sobre representaciones de imagen.

Modelo B — Arquitectura alternativa: puede implementarse cualquier arquitectura distinta basada en literatura científica o un diseño propio fundamentado académicamente. Entre las referencias sugeridas se encuentran **EfficientNet**, **MobileNet**, **Inception**, **DenseNet** y **ResNet**. Si se propone un modelo propio, debe explicarse claramente la motivación del diseño, las decisiones tomadas respecto a profundidad, ancho de los bloques, tipo de bloques, funciones de activación y regularización, y cómo estas decisiones afectan la capacidad de generalización y la eficiencia computacional. Recuerde que la selección del modelo debe considerar el contexto de despliegue en un dispositivo móvil, donde los recursos computacionales son limitados.

Entrenamiento

Se deben entrenar **4 modelos base**, combinando los dos modelos con los dos conjuntos de datos:

- Modelo A / Datos crudos
- Modelo A / Datos aumentados
- Modelo B / Datos crudos
- Modelo B / Datos aumentados

Para cada una de estas combinaciones se deben realizar **al menos 3 entrenamientos con distintos hiperparámetros** (tasa de aprendizaje, tamaño de batch, número de épocas, optimizador, entre otros), resultando en un mínimo de **12 modelos entrenados** en total. Para cada entrenamiento se deben mostrar las curvas de *loss* y métricas tanto del conjunto de entrenamiento como del de validación, con el fin de verificar la ausencia de overfitting o underfitting. Cualquier problema de este tipo debe identificarse y justificarse por escrito en el informe.

Durante el entrenamiento se recomienda el uso de Google Colab en caso de no contar con GPU local, subiendo el dataset a Google Drive para accederlo desde el entorno de ejecución.

Evaluación y Comparación de Modelos

Para la comparación de los modelos entrenados se debe utilizar la herramienta **Weights & Biases (WandB)**, que permite trazar y visualizar en tiempo real múltiples aspectos del proceso de entrenamiento. Las métricas que deben registrarse

y compararse incluyen: *loss* de entrenamiento y validación, *accuracy*, F1-Score y matriz de confusión.

El proceso de selección del mejor modelo se realiza en dos etapas: primero se selecciona el mejor modelo dentro de cada combinación (A/Crudo, A/Aumentado, B/Crudo, B/Aumentado) con base en su desempeño sobre el conjunto de test. Posteriormente, se comparan entre sí los cuatro mejores modelos seleccionados para determinar el modelo final que se desplegará en la aplicación móvil. Esta comparación debe estar documentada con una tabla de resultados y un análisis cualitativo en el informe.

Exportación a Móvil con ONNX

Una vez identificado el mejor modelo, se debe preparar para su despliegue móvil siguiendo estos pasos:

- **Congelar el modelo:** colocar el modelo en modo de inferencia y conservar sus pesos entrenados en la configuración final seleccionada.
- **Definir el pipeline de entrada:** documentar claramente el formato del audio de entrada, frecuencia de muestreo, tipo de preprocesamiento aplicado y la forma esperada del tensor de entrada al modelo.
- **Exportar a ONNX:** exportar el modelo al formato `.onnx` y verificar que las entradas y salidas del archivo generado coincidan con las del modelo original, comparando predicciones sobre ejemplos de prueba conocidos.
- **Integrar en la aplicación:** cargar el archivo `.onnx` en la aplicación móvil mediante **ONNX Runtime Mobile**, asegurando que el preprocesamiento reproducido en la app sea idéntico al utilizado durante el entrenamiento, y que la salida del modelo se transforme correctamente en acciones dentro de la interfaz de usuario.

Entrega

La entrega consiste en un archivo `.zip` que contenga:

- Código fuente en $\text{\LaTeX}$ con plantilla IEEE.
- PDF del informe con visualizaciones e interpretación de resultados.
- Jupyter Notebook con el código completo del modelo.
- Código fuente de la aplicación móvil integrada con los comandos de IA.

El informe será evaluado en contenido y verificado contra los notebooks y el desempeño de la aplicación. Un informe desordenado o sin estructura adecuada puede considerarse incompleto y afectar cualquiera de las rúbricas.

Cuadro III
RÚBRICA DE EVALUACIÓN DEL PROYECTO I

Criterio	Puntaje Máx.
Diseño y arquitectura Modelo A	5
Diseño y arquitectura Modelo B	10
Selección de técnicas de aumentación	10
Entrenamiento A/Crudo y A/Aumentado	15
Entrenamiento B/Crudo y B/Aumentado	15
Métricas y selección mejor modelo A (test)	10
Métricas y selección mejor modelo B (test)	10
Comparación de modelos finales	10
Integración en aplicación móvil	15
Total	100